

ZYVORAI LABS · CUSTOMER DOCUMENTATION

Veyron

Page-by-Page Product Manual

2026-09-10

Page-by-page guides

Each guide follows: Purpose → When to use it → How to get there → What you can do → Related pages.

Every route is also listed in the [complete page index](#).

Compute

Page	What it covers
GPUs	Per-node GPU inventory — passthrough/MIG/vGPU classification and allocatable counts.
Hosts	Kubernetes node inventory — status, kernel, uptime, cordon/uncordon, reboot.
Images & ISOs	ContainerDisk/ISO image catalog plus CDI DataSources, with publish/delete.
Migrations	Live-migration job inventory and status.
Pods	Pod inventory across all namespaces, including virt-launcher pods, with logs and delete.
Template Foundry	Template Foundry — the built-in OS template catalog plus VMProfiles, with clone/delete.
Virtual machines	VirtualMachine inventory — create, start/stop/pause/restart, migrate, snapshot, clone, and

Overview

Page	What it covers
ConsoleHub	ConsoleHub is a one-click launcher for live consoles into your running guests — no need to open
Mission Control	Mission Control is the console's home page — a full-bleed fleet summary that answers "is
Monitoring	Monitoring shows how much headroom the cluster has right now, and which KubeVirt/CDI platform
Topology	Topology shows the cluster as a map of nodes and edges, pulled live from the topology API — a

Security

Page	What it covers
Alerts	Active Kubernetes and KubeVirt alerts — real cluster Events (scheduling failures, crashed VMs,
Security	Security detections surfaced by the SOC (security operations) pipeline — a dedicated feed for

Storage Network

Page	What it covers
Atlas	Ceph-backed (and NFS/ZFS-backed) storage volumes and snapshots surfaced through the Atlas storage control-plane integration.
Backups	Backup job inventory, with run, restore, and delete.
DR & Velero	Cluster backup and failback controls via the Velero integration, dry-run friendly where the API allows.
PacketWolf	Health of the PacketWolf/Cilium network-intelligence integration.
Networks	NetworkAttachmentDefinition (Multus) inventory, with create and delete.

Page	What it covers
Storage	PersistentVolumeClaim inventory plus a StorageClass/storage-pool summary, with resize and delete.
Snapshots	VM disk snapshot inventory, with restore and delete.

System

Page	What it covers
Settings	Console theme, cluster self-healing policy, and the status of this deployment's optional

21 guides. Regenerate: `node scripts/customer-docs/generate-guide-index.mjs` .

GPUs

Purpose

Per-node GPU inventory — passthrough/MIG/vGPU classification and allocatable counts.

When to use it

- Checking GPU capacity before creating a GPU-attached VM
- Confirming which nodes advertise which GPU product/type before scheduling

How to get there

- Left rail: **Compute** → **GPUs**
- Use the top-bar **Search** to jump here directly
- Toggle **list/grid** view with the icon next to the page title

What you'll see

Columns (list view):

Column	Shows
Node	The node advertising the GPU resource
Status	Availability state
Product	GPU product name
Alloc	Allocatable count on that node
Type	Passthrough / MIG / vGPU classification
Memory	GPU memory

On a cluster with no GPU hardware installed, this page shows an empty "No gpus" state — that's expected, not an error.

Select a row to open the **Inspector** (right panel):

- Product
- Type
- Resources
- Memory
- Allocatable

What you can do

This is a read-only inventory page — no row actions. To attach a GPU to a VM, set it when creating the VM (see [Virtual machines](#)); the create form rejects the request up front if no node currently advertises the requested GPU resource.

Related pages

- [Page index](#)
- [Getting started](#)

- [Virtual machines](#)
 - [Hosts](#)
-

Hosts

Purpose

Kubernetes node inventory — status, kernel, uptime, cordon/uncordon, reboot.

When to use it

- Checking which nodes are Ready before scheduling more VMs
- Taking a node out of rotation for maintenance (cordon) and bringing it back (uncordon)

How to get there

- Left rail: **Compute** → **Hosts**
- Use the top-bar **Search** to jump here directly
- Toggle **list/grid** view with the icon next to the page title

What you'll see

Columns (list view):

Column	Shows
Name	Node name
Status	Ready / Cordoned / NotReady
CPU	CPU utilization meter
Memory	Memory utilization meter
VMs	Number of VMs currently on this node
Kernel	Kernel version
Uptime	Node age

CPU/Memory utilization and the VMs count require live metrics wiring the platform API doesn't expose yet — on a cluster without that wired up, those columns legitimately show `0% / -` rather than real usage (verified against `GET /api/v1/nodes`, which currently returns capacity/allocatable and version fields but no per-node usage or VM-count fields). This is a backend gap, not a display bug.

Select a row to open the **Inspector** (right panel):

- Kernel
- Cores
- Memory
- VMs
- Uptime

What you can do

- **cordon** — mark the node unschedulable (existing VMs keep running, no new ones land here)
- **uncordon** — make the node schedulable again
- **reboot** — reboot the node

Related pages

- [Page index](#)
 - [Getting started](#)
 - [Virtual machines](#)
 - [Monitoring](#)
-

Images & ISOs

Purpose

ContainerDisk/ISO image catalog plus CDI DataSources, with publish/delete.

When to use it

- Browsing golden images available to boot VMs from
- Publishing an uploaded image as a versioned golden image (promotes it to a DataSource other VMs can clone from)

How to get there

- Left rail: **Compute** → **Images & ISOs**
- Use the top-bar **Search** to jump here directly
- Toggle **list/grid** view with the icon next to the page title

What you'll see

Columns (list view):

Column	Shows
Name	Image name
Type	Image kind (ISO, qcow2, ...)
Size	Image size
OS	Guest OS the image targets
Added	Age since the image was added

On a cluster with nothing uploaded yet, this page shows an empty "No images" state — that's expected, not an error.

Select a row to open the **Inspector** (right panel):

- Type
- Size
- OS
- Source
- Namespace

A second table, **DataSources**, lists the CDI DataSource objects images publish into:

Column	Shows
Name	DataSource name
Namespace	Owning namespace
Source	Provisioning source
Size	Total size

What you can do

- **publish** — promote a successfully-uploaded image into a versioned golden image + DataSource that VMs can clone from
- **delete** — remove the image
- The **+ New** button in the top bar opens the upload form

Related pages

- [Page index](#)
 - [Getting started](#)
 - [Virtual machines](#)
 - [Storage](#)
-

Migrations

Purpose

Live-migration job inventory and status.

When to use it

- Watching the progress of a live migration you just started from the Virtual machines page
- Checking why a migration job didn't complete

How to get there

- Left rail: **Compute** → **Migrations**
- Use the top-bar **Search** to jump here directly
- Toggle **list/grid** view with the icon next to the page title

What you'll see

Columns (list view):

Column	Shows
ID	Migration job name
Status	Job state
VM	The VM being migrated
Source	Source node
Target	Target node
Progress	Migration progress
Started	Age since the job started

On a single-node cluster this page is always empty — live migration needs a second schedulable node with matching capacity, so there's nothing to migrate to (see the "Live migration" tile on [Monitoring](#), which reports `no` under those conditions).

Select a row to open the **Inspector** (right panel):

- VM
- Source
- Target
- Type
- Progress

What you can do

- **delete** — remove a completed or stale migration job's record

Migrations themselves are started from a VM row's **migrate** action on the [Virtual machines](#) page, not from here.

Related pages

- [Page index](#)
 - [Getting started](#)
 - [Virtual machines](#)
 - [Monitoring](#)
-

Pods

Purpose

Pod inventory across all namespaces, including virt-launcher pods, with logs and delete.

When to use it

- Checking the platform's own component pods (CDI, cert-manager, Cilium, ...) alongside VM virt-launcher pods, in one place
- Pulling logs off a crashing or restarting pod
- Confirming restart counts before escalating a VM issue

How to get there

- Left rail: **Compute** → **Pods**
- Use the top-bar **Search** to jump here directly
- Toggle **list/grid** view with the icon next to the page title

What you'll see

Columns (list view):

Column	Shows
Name	Pod name
Status	Phase (Running, Pending, ...)
Namespace	Owning namespace
Ready	Ready container count
Restarts	Restart count
Node	Node the pod is scheduled on
Age	Pod age

The Ready column currently always shows `—`: `GET /api/v1/pods` returns `name/namespace/phase/node_name/ip/containers/restarts/age`, but no container-readiness field, so there's nothing for the UI to show yet. This is a backend gap, not a display bug — Status and Restarts are accurate.

Select a row to open the **Inspector** (right panel):

- Namespace
- Node
- Image
- Restarts

What you can do

- **logs** — view the pod's container logs
- **delete** — delete the pod (a controller-managed pod, like a virt-launcher, gets recreated)

Related pages

- [Page index](#)
 - [Getting started](#)
 - [Virtual machines](#)
 - [Hosts](#)
-

Template Foundry

Purpose

Template Foundry — the built-in OS template catalog plus VMProfiles, with clone/delete.

When to use it

- Picking an OS template when creating a VM
- Checking a template's default sizing (vCPU/memory/disk) before basing a VM on it
- Browsing VMProfiles (named sizing presets) available to the catalog

How to get there

- Left rail: **Compute** → **Template Foundry**
- Use the top-bar **Search** to jump here directly
- Toggle **list/grid** view with the icon next to the page title

What you'll see

Columns (list view):

Column	Shows
Name	Template name (e.g. <code>almalinux</code> , <code>ubuntu-22.04</code>)
OS	Guest OS family
vCPU	Default vCPU count
Memory	Default memory, in GiB
Disk	Default disk size, in GB
Used by	Number of VMs created from this template

Sizing comes from whichever backend source answers first: the built-in template registry (which carries real vCPU/memory/disk presets) or, when your cluster has synced a VMTemplate CRD catalog (`veyron catalog sync`), that catalog instead — which only carries family/description/tags, no sizing. When the catalog source wins, vCPU/Memory/Disk correctly show `–` rather than a misleading `0` .

Select a row to open the **Inspector** (right panel):

- OS
- Cloud-init
- Used by
- Tags

A second table, **VMProfiles**, lists named sizing presets:

Column	Shows
Name	Profile name
Family	OS family the profile targets
vCPU	vCPU count

Column	Shows
Memory	Memory

What you can do

- **clone** — duplicate a template
- **delete** — remove a template
- The **+ New** button in the top bar opens the create form

Related pages

- [Page index](#)
 - [Getting started](#)
 - [Virtual machines](#)
-

Virtual machines

Purpose

VirtualMachine inventory — create, start/stop/pause/restart, migrate, snapshot, clone, and inspect KubeVirt VMs.

When to use it

- Your daily home base for the VM fleet: power state, sizing, and network address at a glance
- Creating a new VM from a template
- Running day-2 lifecycle actions (stop, pause, snapshot, clone) on an existing VM

How to get there

- Left rail: **Compute** → **Virtual machines**
- Use the top-bar **Search** to jump here directly
- Toggle **list/grid** view with the icon next to the page title

What you'll see

Columns (list view):

Column	Shows
Name	VM name
Status	Power/health state (Running, Stopped, Starting, ...)
vCPU	Allocated virtual CPUs
Memory	Allocated memory, in GiB
Disk	Root disk size, in GB
Host	The node the VM's virt-launcher pod is running on
IP	Guest network address
Uptime	Age since the VM last started

Select a row to open the **Inspector** (right panel):

- Guest OS
- Host
- IP
- Uptime
- Namespace

What you can do

- **start** — power on a stopped VM
- **stop** — graceful power-off
- **restart** — graceful restart (stop, then start)
- **pause** — freeze the running VM in place without powering off

- **unpause** — resume a paused VM
- **migrate** — live-migrate to another node (needs a second schedulable node with matching capacity)
- **console** — open a VNC/serial console session
- **snapshot** — take a point-in-time disk snapshot
- **clone** — create a new VM from this one
- **delete** — remove the VM
- The **+ New** button in the top bar opens the create-VM form (name + template)

Related pages

- [Page index](#)
 - [Getting started](#)
 - [Template Foundry](#)
 - [Snapshots](#)
-

ConsoleHub

Purpose

ConsoleHub is a one-click launcher for live consoles into your running guests — no need to open the Virtual machines page and find the console action on a specific row first.

When to use it

- You know a VM is running and just want its display, fast.
- You want to see every guest with an open console option in one grid, instead of scanning a table.

How to get there

- Left rail: **Overview** → **ConsoleHub**
- This is a full-bleed page — the left rail and Inspector auto-collapse while you're on it. Click the panel-toggle icon (top-left, next to the Veyron wordmark) to bring the rail back if you need it without leaving the page.

What you'll see

- If no VMs are currently running: an empty state ("No running guests — start a machine, then open its console from here") with a **Create a machine** button.
- If VMs are running: a card grid, one card per running VM, showing its name, status, guest OS, IP address, and an **Open** button.

What you can do

- Click **Open** on any card to open a live display (VNC/serial) session for that VM.
- Click **Create a machine** from the empty state to open the new-VM form.

Related pages

- [Page index](#)
 - [Getting started](#)
 - [Virtual machines](#)
-

Mission Control

Purpose

Mission Control is the console's home page — a full-bleed fleet summary that answers "is everything OK?" at a glance, with one-click paths into creating a machine or browsing your existing VMs and hosts.

When to use it

- As your landing page after logging in — check fleet health before diving into a specific resource.
- To quickly launch a new VM without navigating to the Virtual machines page first.
- To jump straight to Virtual machines or Hosts from a single click.

How to get there

- Left rail: **Overview → Mission Control**
- This is a full-bleed page — the left rail and Inspector auto-collapse while you're on it. Click the panel-toggle icon (top-left, next to the Veyron wordmark) to bring the rail back if you need it without leaving the page.

What you'll see

- A dark hero card with the Veyron wordmark, a one-word status line ("Operational." or an attention state), and a summary sentence — e.g. "2 of 2 machines running on 1 host. Nothing needs a look."
- A **Create a machine** pill button, plus **Virtual machines** and **Hosts** quick links, all in the hero.
- A stats strip below the hero: machine count, host count, free memory, and whether live migration is available on this cluster.
- A 5-dot colorway strip ("tone swatch") showing the accent colors used across the console's nav sections.

What you can do

- Click **Create a machine** to open the new-VM form.
- Click **Virtual machines** or **Hosts** to jump to those pages.

Related pages

- [Page index](#)
 - [Getting started](#)
 - [Virtual machines](#)
 - [Hosts](#)
-

Monitoring

Purpose

Monitoring shows how much headroom the cluster has right now, and which KubeVirt/CDI platform versions are actually installed — the two questions you ask before sizing a new VM or planning an upgrade.

When to use it

- Before creating a large VM, to check there's enough free CPU/memory.
- To confirm whether live migration is available on this cluster.
- To check the installed KubeVirt/CDI versions without SSHing into a node.

How to get there

- Left rail: **Overview** → **Monitoring**
- This is a full-bleed page — the left rail and Inspector auto-collapse while you're on it. Click the panel-toggle icon (top-left, next to the Veyron wordmark) to bring the rail back if you need it without leaving the page.

What you'll see

- A hero with a **Refresh** button.
- A stats strip: **CPU free** (cores), **Memory free** (GiB), **Nodes** (count), and whether **Live migration** is supported on this cluster.
- A "Platform stack" section listing each installed component (currently `kubevirt` and `cdi`) as a version + deployment phase, e.g. `v1.9.0 · Deployed`.

What you can do

- Click **Refresh** to re-pull capacity headroom and platform versions.

Related pages

- [Page index](#)
 - [Getting started](#)
 - [Hosts](#)
 - [Topology](#)
-

Topology

Purpose

Topology shows the cluster as a map of nodes and edges, pulled live from the topology API — a quick visual sanity check of the cluster's shape.

When to use it

- To confirm how many nodes are in the cluster and that they're all Ready.
- To check for topology edges (relationships between nodes) when diagnosing placement or networking issues.

How to get there

- Left rail: **Overview** → **Topology**
- This is a full-bleed page — the left rail and Inspector auto-collapse while you're on it. Click the panel-toggle icon (top-left, next to the Veyron wordmark) to bring the rail back if you need it without leaving the page.

What you'll see

- A hero with a **Refresh** button.
- A **Nodes** section listing each node's readiness status, name, and kind (e.g. "Ready `nldw4-4-16-36` Node").
- An **Edges** section listing topology edges between nodes. On a single-node cluster this is typically empty (0 edges).

What you can do

- Click **Refresh** to re-pull the topology map.

Related pages

- [Page index](#)
 - [Getting started](#)
 - [Monitoring](#)
 - [Hosts](#)
-

Alerts

Purpose

Active Kubernetes and KubeVirt alerts — real cluster Events (scheduling failures, crashed VMs, failed readiness probes, manifest-apply errors) surfaced as an actionable list.

When to use it

- Something looks wrong with a VM or the cluster and you want to know why
- You want a single place to triage "what's currently firing" before digging into a specific page
- You're about to acknowledge/close out a known, already-handled issue

How to get there

- Left rail: **Security** → **Alerts**
- Use the top-bar **Search** to jump here directly
- Toggle **list/grid** view with the icon next to the page title

What you'll see

Columns (list view):

Column	Shows
Name	The Kubernetes reason string, e.g. <code>FailedScheduling</code> , <code>FailedCreate</code> , <code>Stopped</code> , <code>Unhealthy</code> , <code>ApplyManifestFailed</code>
Status	<code>firing</code> while the underlying condition is still active
Severity	e.g. <code>warning</code>
Message	The real Event message — for example <code>0/1 nodes are available: 1 node(s)...</code> or <code>The VirtualMachineInstance crashed.</code>
Source	The reporting component, e.g. <code>default-scheduler</code> , <code>virt-handler</code> , <code>kubelet</code> , <code>daemonset-controller</code> , <code>deploy</code>
Fired	When the event was last reported

These are genuine live cluster Events — not synthetic UI state. An alert here means something in Kubernetes or KubeVirt actually reported that condition.

Select a row to open the **Inspector** (right panel):

- Severity
- Message (full, unclipped)
- Source
- Fired (exact timestamp)

The Inspector also has an **Events** tab showing related recent events for that alert (often "No recent events" if the underlying condition hasn't recurred).

What you can do

- **resolve** — mark the alert handled and clear it from the active list. This doesn't change anything in the cluster itself; it only dismisses the alert here. If the underlying condition (e.g. a pod still failing to schedule) is still happening, a fresh alert can reappear.

Related pages

- [Page index](#)
 - [Getting started](#)
 - [Security](#)
-

Security

Purpose

Security detections surfaced by the SOC (security operations) pipeline — a dedicated feed for security-relevant findings, separate from the general [Alerts](#) page.

When to use it

- You want a security-focused view rather than general operational alerts
- You're triaging findings from Veyron's SOC detection rules or an integrated SIEM export
- You're checking whether anything security-relevant has fired before a release or audit

How to get there

- Left rail: **Security** → **Security**
- Use the top-bar **Search** to jump here directly
- Toggle **list/grid** view with the icon next to the page title

What you'll see

Columns (list view):

Column	Shows
Name	The detection name/rule that fired
Status	Detection status
Severity	Detection severity
Message	The detection's description
Source	Which detector/collector produced it
Seen	When it was last observed

On a healthy cluster with no active detections, this page shows an empty state ("No detections"). That's the normal, expected state — it means nothing has tripped a security detection rule, not that the page is broken.

Select a row to open the **Inspector** (right panel):

- Severity
- Message
- Source
- Seen

What you can do

- **ack** — acknowledge a detection, marking it reviewed.

Related pages

- [Page index](#)
- [Getting started](#)
- [Alerts](#)

Atlas

Purpose

Ceph-backed (and NFS/ZFS-backed) storage volumes and snapshots surfaced through the Atlas storage control-plane integration.

When to use it

- See every storage volume Atlas manages across its backends (Ceph RBD, NFS, ZFS) in one place
- Check which volume backs a given VM's PVC, and which pool/backend it lives on
- Review Atlas-level snapshots (separate from KubeVirt's native `VolumeSnapshot` objects)

How to get there

- Left rail: **Storage & network** → **Atlas**
- Use the top-bar **Search** to jump here directly
- Toggle **list/grid** view with the icon next to the page title

This page only shows data when Atlas is configured (`VEYRON_ATLAS_URL` set on the Veyron API deployment). If it isn't configured, the table is empty and [Settings](#) shows the Atlas integration as not configured.

What you'll see

Columns (list view):

Column	Shows
Name	Volume name (an export path for NFS, a pool/name pair for RBD, a dataset path for ZFS)
Status	Volume state (e.g. available)
Namespace	Kubernetes namespace, when this volume backs a PVC
Size	Total volume size
Backend	The storage pool/backend id this volume lives on
PVC	The Kubernetes PVC name this volume backs, when applicable

Not every volume is tied to a Kubernetes object — NFS and ZFS volumes exported for general use may show no Namespace/PVC, while a Ceph RBD volume backing an in-cluster PVC shows both.

Select a row to open the **Inspector** (right panel):

- Namespace
- Backend
- PVC
- Size

Secondary table

Below the main table, **Atlas snapshots** lists Atlas-level snapshots taken of these volumes: name, status, source volume, size, and age.

What you can do

This page is read-only inventory — there are no row actions here. To take a Ceph-backed snapshot of a specific VM's disk, open that VM on the [Virtual machines](#) page and use the **Ceph snap** action in its Inspector panel; that action calls Atlas directly for the selected VM.

Related pages

- [Page index](#)
 - [Getting started](#)
 - [Storage](#)
 - [Virtual machines](#)
-

Backups

Purpose

Backup job inventory, with run, restore, and delete.

When to use it

- Check whether a scheduled backup actually ran and succeeded
- Kick off an ad-hoc backup before a maintenance window
- Restore a VM (or its data) from a prior backup

How to get there

- Left rail: **Storage & network** → **Backups**
- Use the top-bar **Search** to jump here directly
- Toggle **list/grid** view with the icon next to the page title

What you'll see

Columns (list view):

Column	Shows
Name	Backup job name
Target	What the backup covers (VM name, or backup destination)
Items	Item count included in the backup
Size	Backup size
Last run	Age since the backup last ran
Status	Backup status (e.g. Healthy)

On a cluster with no backups configured yet, this page shows an empty state: "No backups — Nothing matches this view."

Select a row to open the **Inspector** (right panel):

- Target
- Schedule
- Retention

What you can do

- **run** — trigger the backup job now
- **restore** — restore from this backup
- **delete** — remove the backup record

This page is the native KubeVirt/VM-snapshot-driven backup listing. For cluster-wide backups via the Velero integration specifically (schedules, off-cluster storage, failback), see [DR & Velero](#) instead — they're two different backup mechanisms that can both be in use.

Related pages

- [Page index](#)
 - [Getting started](#)
 - [DR & Velero](#)
 - [Snapshots](#)
-

DR & Velero

Purpose

Cluster backup and failback controls via the Velero integration, dry-run friendly where the API allows.

When to use it

- Confirm Velero is reachable before relying on cluster-wide backups
- Trigger a Velero backup ahead of a risky cluster change
- Restore from a Velero backup, or fail back a namespace/cluster after DR

How to get there

- Left rail: **Storage & network** → **DR & Velero**
- This is a full-bleed page — the left rail and Inspector auto-collapse while you're on it. Click the panel-toggle icon (top-left, next to the Veyron wordmark) to bring the rail back if you need it without leaving the page.

What you'll see

A hero header ("Disaster Recovery — DR & Velero — Cluster backup and failback controls. Dry-run friendly where the API allows.") with a **Refresh** button, followed by a status section showing whether **Velero** is reachable, and three action cards:

- **Create Velero backup** — dry-run when the API supports it
- **Restore from backup** — prompts you for a backup name
- **DR failback** — confirms before mutating anything

What you can do

- **Refresh** — re-check Velero reachability
- **Create Velero backup** — trigger a new backup
- **Restore from backup** — restore a named backup
- **DR failback** — fail back after a disaster-recovery event (destructive; the console asks for confirmation first)

This page is Velero's cluster-level backup integration — distinct from the VM-snapshot-driven [Backups](#) page, which lists native KubeVirt backup jobs instead. Velero needs an S3-compatible object-storage endpoint configured on the backend (this lab uses Rook-Ceph's RGW gateway for that).

Related pages

- [Page index](#)
 - [Getting started](#)
 - [Backups](#)
 - [Snapshots](#)
-

PacketWolf

Purpose

Health of the PacketWolf/Cilium network-intelligence integration.

When to use it

- Confirm PacketWolf is reachable before relying on network-intelligence data elsewhere
- Diagnose why network-brain data isn't showing up

How to get there

- Left rail: **Storage & network** → **PacketWolf**
- This is a full-bleed page — the left rail and Inspector auto-collapse while you're on it. Click the panel-toggle icon (top-left, next to the Veyron wordmark) to bring the rail back if you need it without leaving the page.

What you'll see

A hero header ("Network Brain — PacketWolf — Cilium / PacketWolf integration health for the console.") with a **Refresh** button, followed by a **Connection** status section showing **Reachable** and a health check row: **Healthy — Health — Probe succeeded**.

This page only reports the *unauthenticated health probe* (`/health`). It can show "Reachable"/"Healthy" even when PacketWolf isn't fully usable: if only a static API key is configured instead of a username/password, the deeper proxied calls (network overview, flow data used elsewhere in the console) will fail with `401 Unauthorized` , because PacketWolf's real auth model is a JWT bearer token obtained via login, not a static key. If network-intelligence data looks missing elsewhere despite this page showing healthy, check that `VEYRON_PACKETWOLF_USERNAME` / `VEYRON_PACKETWOLF_PASSWORD` are set (not just `VEYRON_PACKETWOLF_API_KEY`).

This page only shows real data when PacketWolf is configured (`VEYRON_PACKETWOLF_URL` set). If it isn't configured, the connection status reflects that.

What you can do

- **Refresh** — re-run the health probe

Related pages

- [Page index](#)
 - [Getting started](#)
 - [Settings](#)
 - [Topology](#)
-

Networks

Purpose

NetworkAttachmentDefinition (Multus) inventory, with create and delete.

When to use it

- Define an additional network a VM can attach to beyond the cluster's default pod network
- Check which NADs exist, their CIDR/VLAN, and how many VMs are attached
- Remove a network definition that's no longer needed

How to get there

- Left rail: **Storage & network** → **Networks**
- Use the top-bar **Search** to jump here directly
- Toggle **list/grid** view with the icon next to the page title

What you'll see

Columns (list view):

Column	Shows
Name	NetworkAttachmentDefinition name
Type	Network type (e.g. bridge, macvlan)
CIDR	Subnet, when defined
VLAN	VLAN id, when defined
Attached	Count of VMs currently attached
Status	Network status

On a cluster with no custom networks defined yet, this page shows an empty state: "No networks — Nothing matches this view."

Select a row to open the **Inspector** (right panel):

- Type
- CIDR
- VLAN
- Gateway
- Namespace

What you can do

- **delete** — remove the network definition (fails safely if VMs are still attached)
- The **+ New** button in the top bar opens a form to create a NetworkAttachmentDefinition

Related pages

- [Page index](#)

- [Getting started](#)
 - [Virtual machines](#)
 - [Topology](#)
-

Storage

Purpose

PersistentVolumeClaim inventory plus a StorageClass/storage-pool summary, with resize and delete.

When to use it

- Check which VM disks are bound to which StorageClass and how full they are
- Confirm a StorageClass exists and which one is the cluster default before creating a VM
- Resize a claim that's running low on space, or clean up an orphaned PVC

How to get there

- Left rail: **Storage & network** → **Storage**
- Use the top-bar **Search** to jump here directly
- Toggle **list/grid** view with the icon next to the page title

What you'll see

Columns (list view):

Column	Shows
Name	PVC name
Status	Bound / Pending / Lost
Namespace	Namespace the PVC lives in
Class	StorageClass name
Size	Requested capacity
Used	Usage meter (needs <code>VEYRON_PROMETHEUS_URL</code> wired for real numbers; otherwise shows an estimate)
Access	Access mode (e.g. ReadWriteOnce)

Select a row to open the **Inspector** (right panel):

- Namespace
- Class
- Volume (bound PersistentVolume name)
- Access mode
- Size

Secondary table

Below the main table, **Storage pools** lists every StorageClass in the cluster: name, class, provisioner, total capacity, a used-capacity meter, and volume count. This is where you confirm which class is marked `(default)` — on a fresh cluster that's usually your CSI driver's block class (e.g. `ceph-rbd`), not `local-path`.

What you can do

- **resize** — grow a PVC's requested capacity (shrinking isn't supported by Kubernetes)

- **delete** — remove a PVC (only safe once nothing references it)
- The **+ New** button in the top bar opens a form to create a standalone PVC

Related pages

- [Page index](#)
 - [Getting started](#)
 - [Snapshots](#)
 - [Atlas](#)
-

Snapshots

Purpose

VM disk snapshot inventory, with restore and delete.

When to use it

- Before a risky change to a VM (OS update, config change), take a snapshot first
- Roll a VM back to a known-good point after something breaks
- Clean up old snapshots that are no longer needed

How to get there

- Left rail: **Storage & network** → **Snapshots**
- Use the top-bar **Search** to jump here directly
- Toggle **list/grid** view with the icon next to the page title

What you'll see

Columns (list view):

Column	Shows
Name	Snapshot name
Machine	The VM this snapshot was taken from
Size	Snapshot size
Taken	Age since the snapshot was created
Status	Snapshot status (e.g. Ready)

On a cluster with no snapshots yet, this page shows an empty state: "No snapshots — Nothing matches this view."

Select a row to open the **Inspector** (right panel):

- Machine
- Size
- Taken

What you can do

- **restore** — roll the source VM back to this snapshot
- **delete** — remove the snapshot

Snapshots here use KubeVirt's native `VolumeSnapshot` flow. If Atlas (Ceph-backed storage) is configured, Ceph-level snapshotting is available separately from a VM's Inspector panel — see [Atlas](#).

Related pages

- [Page index](#)
- [Getting started](#)
- [Storage](#)

- [Atlas](#)
-

Settings

Purpose

Console theme, cluster self-healing policy, and the status of this deployment's optional integrations (PacketWolf, Atlas, SSO) and installed platform versions.

When to use it

- Switching between light and dark theme
- Turning on/testing self-healing for degraded VMs
- Checking whether an optional integration (PacketWolf, Atlas storage, SSO) is actually wired up
- Confirming the installed KubeVirt/CDI versions on this cluster

How to get there

- Left rail: **System** → **Settings**
- Use the top-bar **Search** to jump here directly
- This is a full-bleed-adjacent page — the left rail stays available; Settings has its own layout rather than a table/Inspector pair

What you'll see

Appearance

- Theme toggle (Light / Dark). Saved locally in your browser, per device — it doesn't sync across browsers or devices.

Cluster

- **Auto-heal degraded machines** — toggle that enables the self-healing policy (restarts Failed/Unknown VMs automatically when on).
- **Run self-healing now** — two buttons: **Dry-run** (reports which VMs would be restarted, changes nothing) and **Heal** (actually restarts unhealthy VMs). Dry-run is the safe default to check first.

Integrations

- One row per optional integration (PacketWolf, Atlas storage, Single sign-on), each with a status pill: **Healthy** when the integration is configured and its probe succeeded, or **Unknown** when it's either not configured (no `VEYRON_*_URL` set for that integration) or the probe hasn't resolved yet. This section loads asynchronously — if you navigate here and check immediately, integrations can briefly show "Unknown" while the status call is still in flight; give it a couple of seconds (or hit refresh) before reading it as a real outage.

About

- Installed platform versions — KubeVirt and CDI operator/observed/target versions and rollout phase (e.g. `v1.9.0 · Deployed`), plus a note that upgrading is a separate, explicitly guarded operation (this page only reports what's installed; it doesn't trigger an upgrade).

What you can do

- Switch theme (Light/Dark)
- Toggle auto-heal on/off
- Trigger a self-healing dry-run or an actual heal pass

- [Check integration health and installed platform versions at a glance](#)

Related pages

- [Page index](#)
 - [Getting started](#)
 - [Admin basics](#)
-

Veyron — Complete page index

Every primary navigable console page.

Generated: 2026-09-10 · 21 routes

Regenerate: `node scripts/customer-docs/generate-page-index.mjs`

Overview

Page	Route	Purpose	Guide
Mission Control	<code>mission</code>	Mission Control — full-bleed fleet summary: machines running, host count, quick create, and per-namespace tone accents.	Open
ConsoleHub	<code>console</code>	ConsoleHub — quick links into VNC/serial console sessions and day-2 actions for running VMs.	Open
Monitoring	<code>monitoring</code>	Capacity headroom (free CPU/memory, node count, live-migration support) and installed KubeVirt/CDI platform versions.	Open
Topology	<code>topology</code>	Cluster map as nodes and edges from the topology API.	Open

Compute

Page	Route	Purpose	Guide
Virtual machines	<code>vms</code>	VirtualMachine inventory — create, start/stop/pause/restart, migrate, snapshot, clone, and inspect KubeVirt VMs.	Open
Hosts	<code>hosts</code>	Kubernetes node inventory — status, kernel, uptime, cordon/uncordon, reboot.	Open
GPUs	<code>gpus</code>	Per-node GPU inventory — passthrough/MIG/vGPU classification and allocatable counts.	Open
Images & ISOs	<code>images</code>	ContainerDisk/ISO image catalog plus CDI DataSources, with publish/delete.	Open
Pods	<code>Pods</code>	Pod inventory across all namespaces, including virt-launcher pods, with logs and delete.	Open
Template Foundry	<code>templates</code>	Template Foundry — the built-in OS template catalog plus VMProfiles, with clone/delete.	Open
Migrations	<code>migrations</code>	Live-migration job inventory and status.	Open

Storage & network

Page	Route	Purpose	Guide
Storage	<code>pvcS</code>	PersistentVolumeClaim inventory plus StorageClass/storage-pool summary, with resize/delete.	Open
Snapshots	<code>snapshots</code>	VM disk snapshot inventory, with restore/delete.	Open
Backups	<code>backups</code>	Backup job inventory, with run/restore/delete.	Open
Atlas	<code>atlas</code>	Ceph-backed storage volumes and snapshots from the Atlas storage control-plane integration.	Open

Page	Route	Purpose	Guide
Networks	networks	NetworkAttachmentDefinition (Multus) inventory, with create/delete.	Open
PacketWolf	network-brain	PacketWolf/Cilium network-intelligence integration health.	Open
DR & Velero	dr	Cluster backup and failback controls (Velero), dry-run friendly where the API allows.	Open

Security

Page	Route	Purpose	Guide
Alerts	alerts	Active Kubernetes/KubeVirt alerts and events, with resolve.	Open
Security	soc	Security detections surfaced by the SOC pipeline, with acknowledge.	Open

System

Page	Route	Purpose	Guide
Settings	settings	Console theme, self-healing policy, and integration status (PacketWolf, Atlas, SSO) for this deployment.	Open

Related

- [Customer docs home](#)
- [Page-by-page guides](#)